

QUESTIONS 10 / 10 completed

AVI Tree- Balance F ▾

AVL TREE- BALANCE FACTOR

Write a C program to display the balance factor of each node in an AVL Tree.

PROGRAM EDITOR

C ▾

Run

Save

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node *left, *right;
7 };
8
9 struct Node* createNode(int data) {
10     struct Node* node = (struct Node*)malloc(sizeof(struct Node));
11     node->data = data;
12     node->left = node->right = NULL;
13     return node;
14 }
15
16 struct Node* insert(struct Node* root, int data) {
17     if (root == NULL)
18         return createNode(data);
19
20     if (data < root->data)
21         root->left = insert(root->left, data);
```

INPUT

6
50 30 20 40 70 60

QUESTIONS 10 / 10 completed

AVL Tree- Insert Ele ▾

AVL TREE- INSERT ELEMENTS

"Write a program to create an AVL Tree and insert the following elements:
50, 30, 70, 20, 40, 60, 80.
Construct the AVL Tree,
Display the tree using
Inorder Traversal, Verify
that the height balance
property is maintained
after each insertion."

PROGRAM EDITOR

C ▾

Run

Save

```
1 #include<stdio.h>
2 #include<stdlib.h>
3
4 struct node{int d,h;struct node*l,*r;};
5
6 int height(struct node*n){return n?n->h:0;}
7 int max(int a,int b){return a>b?a:b;}
8
9 struct node*new(int x){
10     struct node*n=malloc(sizeof(struct node));
11     n->d=x;n->h=1;n->l=n->r=NULL;return n;
12 }
13
14 struct node*right(struct node*y){
15     struct node*x=y->l,*t=x->r;
16     x->r=y;y->l=t;
17     y->h=max(height(y->l),height(y->r))+1;
18     x->h=max(height(x->l),height(x->r))+1;
19     return x;
20 }
21
```

INPUT

7
50 30 70 20 40 60 80

QUESTIONS 10 / 10 completed

AVL Tree- Balancing

AVL TREE- BALANCING

Insert the elements: 30, 20, 10 Use C Language to implement the Process. Show the AVL Tree before balancing. Perform the required rotation. Display the balanced AVL Tree. Calculate the balance factor of each node.

PROGRAM EDITOR

C

Run

Save

```
1 #include<stdio.h>
2 #include<stdlib.h>
3
4 struct node{int d;struct node*l,*r;};
5
6 struct node*new(int x){
7     struct node*n=malloc(sizeof(struct node));
8     n->d=x;n->l=n->r=NULL;return n;
9 }
10
11 void pre(struct node*t){
12     if(t){printf("%d ",t->d);pre(t->l);pre(t->r);}
13 }
14
15 struct node*right(struct node*y){
16     struct node*x=y->l;y->l=x->r;x->r=y;return x;
17 }
18
19 int main(){
20     struct node*r=new(30);
21     r->l=new(20);r->l->l=new(10);
```

INPUT

30 20 10

QUESTIONS 10 / 10 completed

AVL Tree -Balance I ▾

AVL TREE - BALANCE NODE

Create an AVL Tree from user-entered values and perform following tasks. Calculate the height of every node. Determine the balance factor of each node. Display nodes that violate AVL property before balancing.

PROGRAM EDITOR

C ▾

Run

Save

```
1 #include<stdio.h>
2 #include<stdlib.h>
3
4 struct node{int d;struct node*l,*r;};
5
6 struct node*in(struct node*t,int x){
7     if(!t){
8         t=malloc(sizeof(struct node));
9         t->d=x;t->l=t->r=NULL;
10    }else if(x<t->d)t->l=in(t->l,x);
11    else t->r=in(t->r,x);
12    return t;
13 }
14
15 int h(struct node*t){
16     if(!t)return 0;
17     int a=h(t->l),b=h(t->r);
18     return (a>b?a:b)+1;
19 }
20
21 void check(struct node*t){
```

INPUT

7
50 30 70 20 40 60 80

QUESTIONS 10 / 10 completed

AVL Tree- Sequenc

AVL TREE-SEQUENCE

Insert the following sorted sequence: 10, 20, 30, 40, 50, 60, 70.

Perform the following operations: 1) Construct a BST. 2) Construct an AVL Tree 3) Compare Height, Number of Rotations.

PROGRAM EDITOR

C



Run

Save

```
1 #include<stdio.h>
2 #include<stdlib.h>
3
4 struct node{int d,h;struct node*l,*r;};
5
6 int H(struct node*n){return n?n->h:0;}
7 int M(int a,int b){return a>b?a:b;}
8
9 struct node*new(int x){
10     struct node*n=malloc(sizeof(struct node));
11     n->d=x;n->h=1;n->l=n->r=NULL;return n;
12 }
13
14 struct node*R(struct node*y){
15     struct node*x=y->l;y->l=x->r;x->r=y;
16     y->h=M(H(y->l),H(y->r))+1;
17     x->h=M(H(x->l),H(x->r))+1;
18     return x;
19 }
20
21 struct node*I(struct node*x){
```

INPUT

7
10 20 30 40 50 60 70

QUESTIONS 10 / 10 completed

Menu-Driven AVL 1 ▾

MENU-DRIVEN AVL TREE

Develop a menu-driven
AVL Tree program
supporting: Insert,
Delete, Search, Display
Inorder Traversal, Display
Height and Exit.

PROGRAM EDITOR

C ▾

Run

Save

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int key;
6     struct Node *left;
7     struct Node *right;
8     int height;
9 };
10
11 int max(int a, int b) {
12     return (a > b) ? a : b;
13 }
14
15 int height(struct Node *N) {
16     if (N == NULL)
17         return 0;
18     return N->height;
19 }
20
21 struct Node* newNode(int key) {
```

INPUT

```
1
10
1
20
1
30
4
5
3
20
2
20
4
6
```

QUESTIONS 10 / 10 completed

AVL Tree- Impleme ▾

AVL TREE- IMPLEMENT ROTATIONS

Construct an AVL Tree
and implement rotations
LL, RR, LR, RL and
balancing using
subprograms and verify
it.

PROGRAM EDITOR

C ▾

Run

Save

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node
5 {
6     int data;
7     struct Node *left, *right;
8     int height;
9 };
10
11 int max(int a, int b)
12 {
13     return (a > b) ? a : b;
14 }
15
16 int height(struct Node *n)
17 {
18     if (n == NULL)
19         return 0;
20     return n->height;
21 }
```

INPUT

3
30 20 10

QUESTIONS 10 / 10 completed

AVL Tree - Deletion ▾

AVL TREE - DELETION

Write a C program to delete a node 88 from an AVL Tree and rebalance the tree if necessary. Use the following elements 77, 22, 99, 55, 88, 33, 11, 66, 44.

PROGRAM EDITOR

C ▾

Run

Save

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int key;
6     struct Node *left, *right;
7     int height;
8 };
9
10 int max(int a, int b) {
11     return (a > b) ? a : b;
12 }
13
14 int height(struct Node *N) {
15     if (N == NULL)
16         return 0;
17     return N->height;
18 }
19
20 struct Node* newNode(int key) {
21     struct Node* node = (struct Node*)malloc(sizeof(struct Node));
```

INPUT

```
9
77 22 99 55 88 33 11
66 44
88
```

QUESTIONS 10 / 10 completed

AVL Tree- Height ▾

AVL TREE- HEIGHT

Write a C program to find the height of an AVL Tree. Create an AVL Tree 50 30 20 40 70 60. Display height after each insertion.

PROGRAM EDITOR

C ▾

Run

Save

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int key;
6     struct Node *left, *right;
7     int height;
8 };
9
10 int max(int a, int b) {
11     return (a > b) ? a : b;
12 }
13
14 int height(struct Node *N) {
15     if (N == NULL)
16         return 0;
17     return N->height;
18 }
19
20 struct Node* newNode(int key) {
21     struct Node* node = (struct Node*)malloc(sizeof(struct Node));
```

INPUT

50 30 20 40 70 60